IF2230 Jarkom
Rangkuman UAS

Transport Layer (End-to-End Protocols)

- Transport layer provides the logical communication between app processes that run on different hosts
- Transport protocols run in end systems:
  - **send** side: breaks app messages into **segments** and passes to network layer
  - **receiver** side: reassembles segments into messages and passes to app layer
- Function of transport layers are:
  - Provide port number to identify application within a host
  - Chop a long stream of data bits into segments and send them off
  - Reassemble the segments at the receiver
  - Open and close a logical connection
  - Provide reliable channel
  - Provide flow control
- The internet offers two transport protocols available to applications, which are **TCP** and **UDP**.

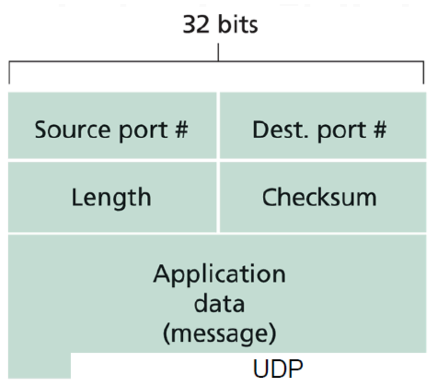


Figure 3.7 ♦ UDP segment structure

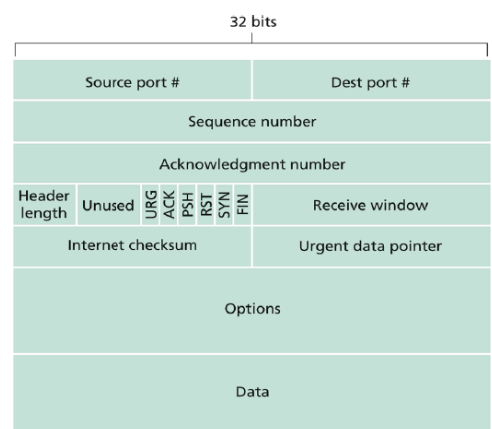


Figure 3.29 ♦ TCP segment structure

	TCP (Transmission Control Protocol)	**UDP (User Datagram Protocol)**
Method	Stream oriented	Datagram oriented
Connection	Connection-oriented (needs handshake to setup and tear down)	Connectionless (no handshake, **just send**)
Reliability	Reliable (no error, no data loss, and in proper order)	Unreliable (no guarantee that the segment will reach the receiving end)

Flow control	Yes, sender won't overwhelm receiver	-
Congestion control	Yes, throttle sender when network overloaded	-
Transmission	only unicast	unicast and multicast
Speed	Slower than UDP	Faster than TCP
Does not provide	timing, minimum bandwidth guarantee	connection setup, reliability, flow control, congestion control, timing, bandwidth guarantee
Used for	web (http), email (stmp), file transfer (ftp), terminal (telnet), etc.	network management (SNMP), routing (RIP), naming (DNS), etc.

- TCP is used by application layer when data integrity and data error is an issue, and also when delay is tolerable

UDP

- no connection, no reliability
- transport layer still need to provide port number
- need the speed but "best effort" service, hence UDP segment may be lost or out of order
- Although UDP is unreliable, u can make it reliable in **software** implementation (performed in application layer instead of transport layer)
- IRL, tftp (trivial file transfer protocol) is a reliable application despite sending data through UDP
- In general, UDP is used by application layer when speed and delay is an issue, when lost of data is not so critical, and reliability of data is not done in transport layer, but in application layer

UDP Checksum

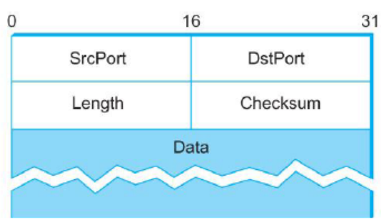
- The goal of checksum is to detect "errors" in transmitted segment

Sender	**Receiver**
- treat segment content as	- compute checksum of received

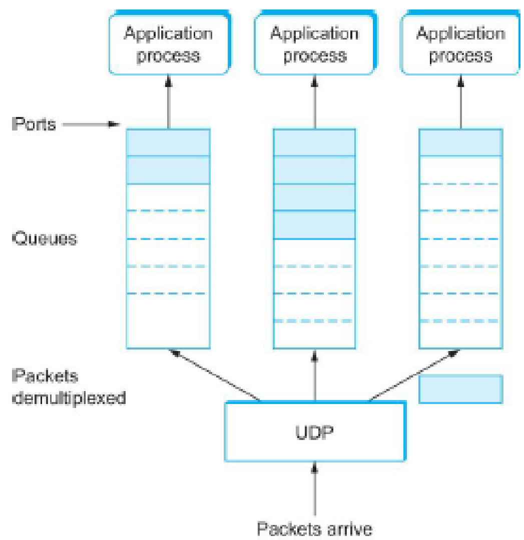
sequence of 16-bit integers - checksum: addition (1's complement sum) of segment contents - sender puts checksum value into UDP checksum field	segment - check if computed checksum equals checksum field value - NO: error detected - YES: no error detected

Simple Demultiplexer (UDP)

- Extends host-to-host delivery service of the underlying network into a process-to-process communication service
- Adds a level of demultiplexing which allows multiple application processes on each host to share the network



Format for UDP header (Note: length and checksum fields should be switched)



UDP Message Queue

Reliable Byte Stream (TCP)

- reliable, connection oriented, byte-stream service

End-to-end issues

- As TCP runs over the internet rather than a point-to-point link, the following issues need to be addressed by the sliding window algorithm
  - TCP supports logical connections between processes that are running on two different computers in the internet
  - TCP connections are likely to have widely different RTT times
  - Packets may get reordered in the internet

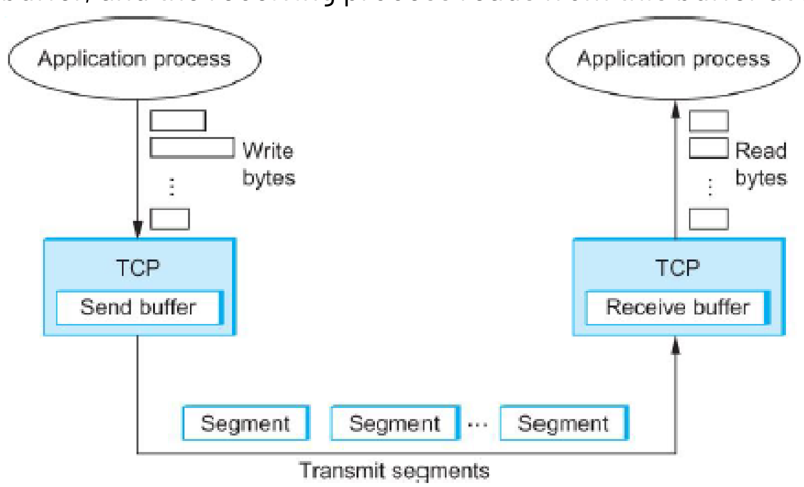
- TCP needs a mechanism using which each side of a connection will learn what resources the other side is able to apply to the connection
- TCP needs a mechanism using which the sending side will learn the capacity of the network.

Flow Control vs Congestion Control

- Flow control involves preventing senders from overrunning the capacity of the receivers
- Congestion control involves preventing too much data from being injected into the network, thereby causing switches or links to become overloaded

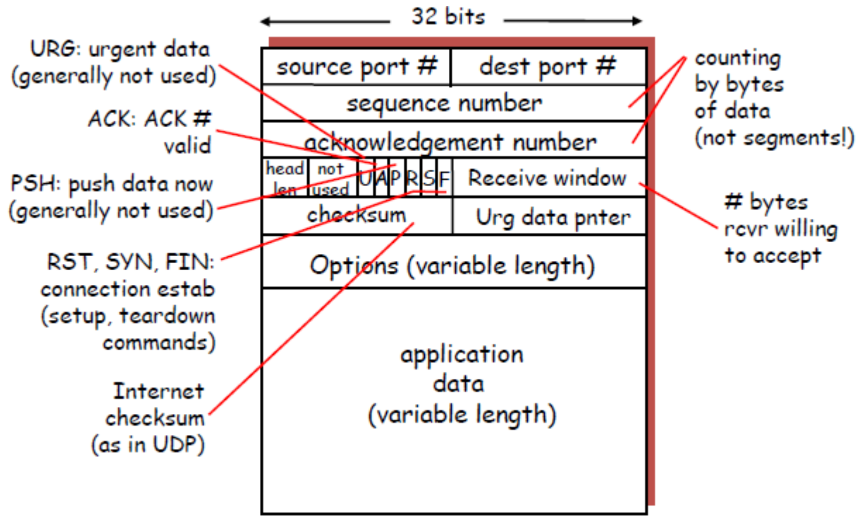
TCP Segment

- TCP is a byte-oriented protocol, which means that the sender writes bytes into a TCP connection and the receiver reads bytes out of the TCP connection
- TCP doesn't transmit individual bytes over the internet.
- The packets exchanged between TCP peers are called **segments** which consists of **TCP header** and **data payload** (the actual data being sent)
- On the **source host**: TCP buffers enough bytes from the sending process to fill a reasonably sized packet and then sends this packet to its peer on the destination host
- On the **destination host**: TCP empties the contents of the packet into a receive buffer, and the receiving process reads from this buffer at its leisure.



How TCP manages a byte stream.

- **TCP header**

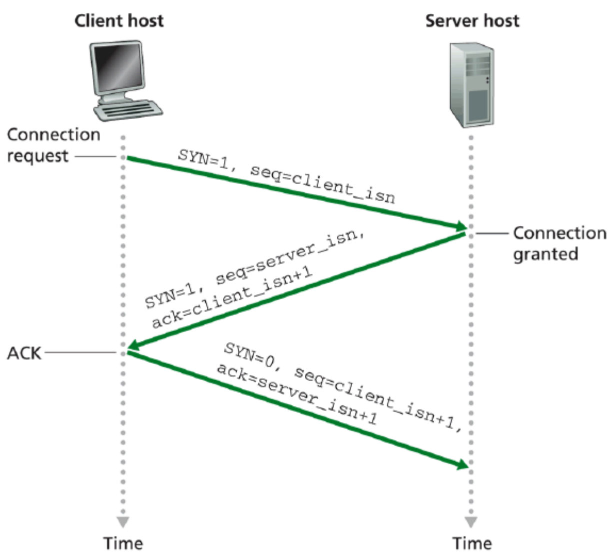


  - **Source port**: identifies the sending port
  - **Destination port**: identifies the receiving port
  - **Sequence number**: If the SYN flag is present then this is the initial sequence number and the first data byte is the sequence number plus 1. Otherwise, then the first data byte is the sequence number
  - **Acknowledgement number**: If the ACK flag is set then the value of this field is the sequence number the sender expects next
  - **Data offset (header length)**: This 4-bit field specifies the size of the TCP header in 32-bit words. The minimum size header is 5 words and the maximum is 15 words thus giving the minimum size of 20 bytes and maximum of 60 bytes. This field gets its name from the fact that it is also the offset from the start of the TCP packet to the data.
  - **Window**: The number of bytes the sender is willing to receive starting from the acknowledgement field value
  - **Reserved**: 4-bit reserved field for future use and should be set to zero.
  - **Checksum**: used for error-checking of the header and data.
  - **URG**: Urgent pointer is valid. If the bit is set, the following bytes contain an urgent message in the range SeqNo <= urgent message <= SeqNo+urgent pointer
  - **ACK**: acknowledgement number is valid
  - **PSH**: push flag. Notification from sender to the receiver that the receiver should pass all data that it has to the application. Used to pass application layer header to application
  - **RST**: reset the connection. The flag causes the receiver to reset the connection. Receiver of a RST terminates the connection and indicates

higher layer application about the reset. Normally means the closing of the network application

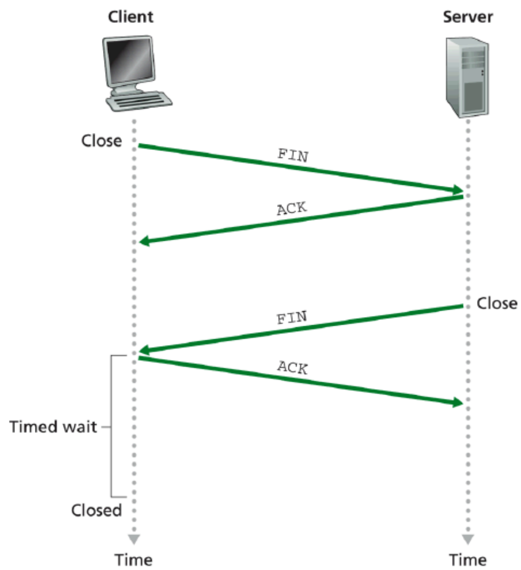
- **SYN**: synchronize sequence numbers. Sent in the first packet when initiating a connection
- **FIN**: sender is finished with sending. used for closing a connection. both sides of a connection may send a FIN

Setting Up A Connection in TCP - Three-way Handshake



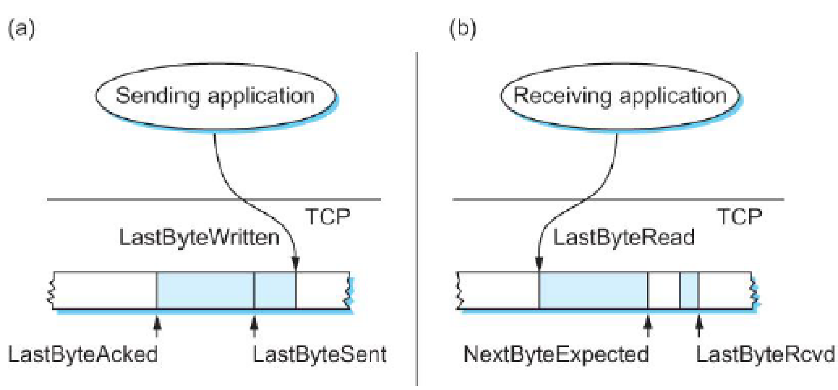
1. Client host sends TCP SYN segment to server
    a. specifies initial seq #
    b. SYN flag is set
    c. no data
2. Server host receives SYN, replies with SYN-ACK segment
    a. server allocates buffers
    b. specifies server initial seq #
3. client receives SYN-ACK, replies with ACK segment, which may contain data

Closing A Connection in TCP



1. client end system sends TCP FIN control segment to server
2. server receives FIN, replies with ACK. Closes connection, sends FIN.

Sliding Window



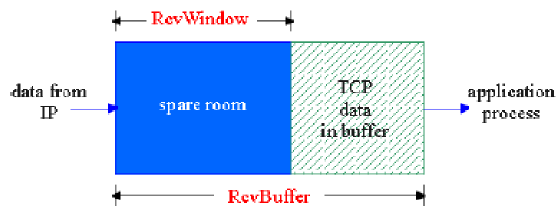
Relationship between TCP send buffer (a) and receive buffer (b).

- TCP's variant of the sliding window algorithm, which serves several purposes:
  - guarantees the reliable delivery of data
  - ensures that data is delivered in order
  - enforces flow control between sender and receiver
- Sending side:
  - LastByteAcked <<= LastByteSent
  - LastByteSent <= LastByteWritten
- Receiving side:

  - LastByteRead < NextByteExpected
  - NextByteExpected <= LastByteReceived + 1

TCP Flow Control

- sender won't overflow receiver's buffer by transmitting too much, too fast
- Receive side of TCP connection has a receive buffer



Suppose TCP receiver discards out-of-order segments
  - spare room in buffer = RcvWindow
    = RcvBuffer - [LastByteRcvd - LastByteRead]
  - Rcvr advertises spare room by including value of RcvWindow in segments
  - Sender limits unACKed data to RcvWindow which guarantees receive buffer doesn't overflow
- app process may be slow at reading from buffer
- speed-matching service: matching the send rate to the receiving app's drain rate
- LastByteRcvd – LastByteRead ≤ MaxRcvBuffer
- AdvertisedWindow = MaxRcvBuffer – ((NextByteExpected – 1) – LastByteRead)
- LastByteSent – LastByteAcked ≤ AdvertisedWindow
- EffectiveWindow = AdvertisedWindow – (LastByteSent – LastByteAcked)
- LastByteWritten – LastByteAcked ≤ MaxSendBuffer
- If the sending process tries to write y bytes to TCP, but (LastByteWritten – LastByteAcked) + y > MaxSendBuffer, then TCP blocks the sending process and does not allow it to generate more data.

Sequence Number Wraparound

- Sequence number wraparound adalah peristiwa ketika nomor urut (*sequence number*) yang digunakan untuk melacak byte data yang dikirim telah mencapai batas nilai maksimumnya dan kembali ter-reset ke angka nol (0).
- TCP menggunakan Sequence Number berukuran 32-bit pada headernya untuk memberikan nomor urut pada setiap byte data yang dikirimkan ⇒ TCP hanya bisa menghasilkan nomor urut dari 0 hingga 2^32 - 1 (4.294.967.295B atau sekitar 4,29 GB)
- TCP menggunakan algoritma *sliding window* untuk mengontrol aliran data. Syarat mutlak dari algoritma ini adalah:

    - **Ruang nomor urut (*sequence number space*) minimal harus dua kali lipat lebih besar dari ukuran jendelanya (*window size*)**.

  Ini penting agar penerima tidak bingung membedakan mana paket dari putaran saat ini dan mana paket dari putaran berikutnya.
- Setiap paket data yang beredar di internet memiliki batas waktu hidup maksimal yang disebut **MSL (*Maximum Segment Lifetime*)**
- MSL umumnya diatur selama **120 detik** ⇒ Jika sebuah paket nyasar atau tertunda di jaringan selama >120 detik, paket itu otomatis dihancurkan
- Bahaya wraparound ⇒ jika *sequence number* berputar kembali ke angka yang sama, sementara paket lama dengan nomor tersebut **masih hidup** (blm lewat 120 detik) di dalam jaringan. Oleh karena itu, **proses wraparound TIDAK BOLEH terjadi dalam waktu kurang dari 120 detik**
- semakin cepat koneksi internet, semakin cepat kuota sequence number 32-bit TCP habis

Bandwidth	Time until Wraparound
T1 (1.5 Mbps)	6.4 hours
Ethernet (10 Mbps)	57 minutes
T3 (45 Mbps)	13 minutes
Fast Ethernet (100 Mbps)	6 minutes
OC-3 (155 Mbps)	4 minutes
OC-12 (622 Mbps)	55 seconds
OC-48 (2.5 Gbps)	14 seconds

Wrap Around Time
= (Total sequence number) / (Bandwidth)
= (2^32) / (Bandwidth)

Time until 32-bit sequence number space wraps around.

Triggering Transmission

- TCP has three mechanism to trigger the transmission of a segment
    - TCP maintains a variable maximum segment size (MSS) and sends a segment as soon as it has collected MSS bytes from the sending process
        - MSS is usually set to the size of the largest segment TCP can send without causing local IP to fragment.
        - MSS: MTU of directly connected network – (TCP header + and IP header)
    - Sending process has explicitly asked TCP to send it
        - TCP supports push operation
    - When a timer fires
        - Resulting segment contains as many bytes as are currently buffered for transmission

- **Silly window syndrome**:

Silly Window Syndrome

  - arises due to poor implementation of TCP, which degrades the TCP performance and makes **data transmission extremely inefficient**. The problem is called so because:
    - it causes the sender window size to shrink to a silly value
    - the window size shrinks to such an extent that the data being transmitted is smaller than TCP header
  - **Penyebab silly window syndrome:**
    - **Sender window transmitting one byte of data repeatedly**
      Suppose only one byte of data is generated by an application. The poor implementation of TCP leads to transmit this small segment of data. Every time the application generates a byte of data, the window transmits it.
    - **Receiver window accepting one byte of data repeatedly**
      Suppose consider the case when the receiver is unable to process all the incoming data. In such case, the receiver will advertise a small window size. The process continues and the window size becomes smaller and smaller. A stage arrives when it repeatedly advertises window size of 1.
  - **Solution: use a timer, transmit when the timer expires**
- **Nagle's Algorithm**
  - **The Goal:** Nagle's algorithm was created to solve the "small packet problem." For example, if you type a single character in an SSH session, you are sending 1 byte of data wrapped in 40 bytes of TCP/IP headers, which is highly inefficient. Nagle's algorithm fixes this by holding back small outgoing

packets and buffering them until either a full-sized packet can be built, or until an Acknowledgment (ACK) for the previously sent data is received.

-
  ```
  When the application produces data to send
      if both the available data and the window >= MSS
          send a full segment
      else
          if there is unACKed data in flight
                  buffer the new data until an ACK arrives
          else
                  send all the new data now
  ```
- **Problem**: The fatal flaw of Nagle's algorithm happens when it interacts with another TCP optimization called Delayed ACK.
  - Delayed ACK tells the receiving computer: "Don't send an ACK immediately. Wait a fraction of a second (typically up to 200ms) to see if you can piggyback the ACK onto some outgoing data."
  - **Deadlock**: The sender (using Nagle) says, "I have a small packet to send, but I am not sending it until I get an ACK." Meanwhile, the receiver (using Delayed ACK) says, "I received the first packet, but I am waiting to send the ACK until I receive more data."
  - **Result**: The two computers end up in a temporary standoff, waiting for each other. The deadlock only breaks when the receiver's Delayed ACK timer finally expires (up to 200ms later). This introduces a severe, noticeable lag spike.

- Karn/Partridge Algorithm
  - **The Goal:** This algorithm was designed to solve the **Retransmission Ambiguity Problem**. If a packet times out and you retransmit it, and then an ACK finally arrives, you have a problem: is that ACK for the *original* delayed packet, or for the *retransmitted* packet? If you guess wrong, your Round Trip Time (RTT) calculations will be completely broken. Karn/Partridge solves this with a strict rule "**Do not update the RTT estimate using ACKs from retransmitted packets.**"
  - **The Problem:** By ignoring the RTT of retransmitted packets, the algorithm creates a dangerous "blind spot" when network conditions suddenly worsen.
    - Imagine a network router suddenly becomes heavily congested, causing delays to spike drastically.
    - Because the network is suddenly much slower than your TCP connection's current RTT estimate, your packets start timing out prematurely, forcing you to retransmit them.

        - Because of Karn's rule, you **ignore all the ACKs** for these retransmissions, meaning your TCP connection is blocked from updating its RTT estimate
        - **Result**: Your TCP connection's internal timer gets "stuck" believing the network is still fast. It completely fails to learn that the network has genuinely slowed down. This can trigger a vicious cycle: you timeout prematurely → retransmit → ignore the ACK → timeout prematurely again
- **Jacobson/Karels Algorithm**
    - **Problem solved**: TCP menggunakan rata-rata sederhana untuk memperkirakan RTT (*Round Trip Time* atau waktu tempuh paket bolak-balik). Aturan lamanya adalah: hitung rata-rata RTT, lalu kalikan dua untuk menentukan batas waktu *Timeout*
    - **The Goal:** TCP harus memperhitungkan seberapa besar fluktuasi (varian) di jaringan tersebut

      Difference = SampleRTT − EstimatedRTT

      EstimatedRTT = EstimatedRTT + (× Difference)

      Deviation = Deviation + (|Difference| − Deviation)

      TimeOut = μ × EstimatedRTT + × Deviation

      – where based on experience, μ is typically set to 1 and is set to 4. Thus, when the variance is small, TimeOut is close to EstimatedRTT; a large variance causes the deviation term to dominate the calculation.

Congestion Control

Resource Allocation Mechanisms

- If the network takes active role in allocating resources, the congestion may be avoided and theres no need for congestion control. But allocating resources with any precision is difficult because resources are distributed throughout the network.
- The issues in resource allocation within a network are categorized into three main areas, the underlying network model, the taxonomy of approaches used, and the criteria for evaluating the allocation's success
    - **Network model constraints**
      Resource allocation takes place in a packet-switched network filled with multiple links and routers, where traffic from competing users can easily create bottlenecks
        - **Potential bottlenecks**:A source might have plenty of capacity on its immediate outgoing link, but its packets can encounter a congested link deeper within the network being shared by many different sources.
        - **Connectionless flows**: while the network is essentially connectionless, datagrams belonging to a specific pair of hosts usually flow through the same set of routers. These flows can be viewed at different granularities, such as host-to-host or process-to-process
        - **Soft state maintenance**: because packets follow a flow, routers may maintain "soft state" information for each flow to make better allocation decisions. This requires the network to balance between a purely connectionless model (no state maintained) and a purely connection-oriented model (hard state maintained)
    - **Taxonomy of allocation approaches**
      Designing a resource allocation mechanism involves choosing how the network and the hosts will interact. The issues here stem from deciding which approach to implement:
        - **Router-centric vs. Host-centric**: In a router-centric approach, the routers take responsibility for deciding which packets to forward or drop, and they inform the hosts of their allowed sending rates. Conversely, in a host-centric approach, the end hosts observe network conditions (like packet loss) and adjust their sending rates themselves

- **Reservation-based vs. Feedback-based**: A reservation-based system requires the host to explicitly ask the network to allocate a specific amount of capacity (buffers or bandwidth) before sending data. A feedback-based system allows the host to send data without a reservation, adjusting its rate based on explicit or implicit feedback from the network
- **Window-based vs. Rate-based**: Systems can regulate traffic by using a window advertisement to reserve buffer space within the network, or they can control the sender's behavior by dictating a specific rate (bits per second) that the network can absorb

○ **Evaluation criteria (trade-offs)**

Once a resource allocation scheme is implemented, it must be evaluated. The core issues here revolve around balancing conflicting goals:

- **Effective resource allocation (throughput vs. delay)**: The two principal metrics for network performance are **throughput** and **delay**. The goal is to achieve high throughput and low delay, but these goals are fundamentally at odds. Allowing maximum packets into the network increases link utilization (throughput) but simultaneously lengthens router queues, causing significant delays. Network designers evaluate this trade-off using the the metric of "power" represented by the formula:

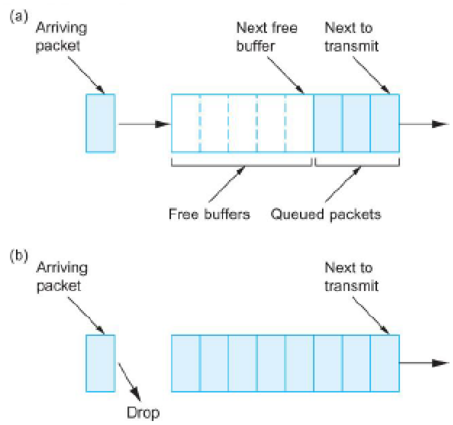
$$Power = \frac{Throughput}{Delay}$$

- **Fair resource allocation**: Effective utilization is not enough; the network must also be fair. Defining fairness is difficult. A reservation-based scheme can create intentional, "controlled unfairness". Without reservations, flows should ideally receive an equal share of bandwidth, though factors like the length of the path the flow takes can complicate what is truly "fair". This fairness can be measured mathematically using Jain's fairness index, which outputs a score between 0 and 1 with 1 representing the greatest fairness.
  - Jain's fairness index is defined as follows. Given a set of flow throughputs ($x_1, x_2, \ldots, x_n$) (measured in consistent units such as bits/second), the following function assigns a fairness index to the flows:

$$f(x_1, x_2, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}$$

Queuing Mechanisms

- Queuing disciplines are mechanisms used within network elements, such as routers, to control the order in which packets are transmitted and to determine which packets are dropped
- There are three main types of queuing disciplines as follows:
    - **FIFO (First In First Out)**:

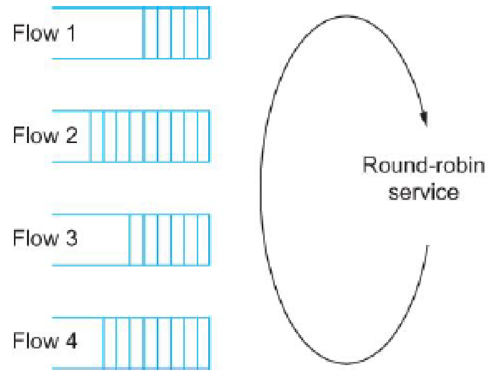


(a) FIFO queuing; (b) tail drop at a FIFO queue.

        - AKA First Come First Served (FCFS), this is a simple approach where the first packet to arrive at a router is the first one to be transmitted
        - FIFO processes packets without considering which flow they belong to or how important they are
        - It is typically paired with a "tail drop" policy, meaning if a packet arrives and the router's buffer is completely full, that newly arriving packet is immediately discarded
        - The presentation emphasizes that FIFO and tail drop are separate concepts; FIFO dictates the transmission order (scheduling discipline), while tail drop dictates which packets are discarded (drop policy)
    - **Priority queue**:
        - A variation of the basic FIFO method where each packet is explicitly marked with a specific priority level, such as within the IP header
        - To implement this, routers maintain multiple separate FIFO queues, dedicating one to each priority class
        - The router will always transmit packets from the highest-priority queue first, only moving to the next priority level when the higher-priority queue is empty

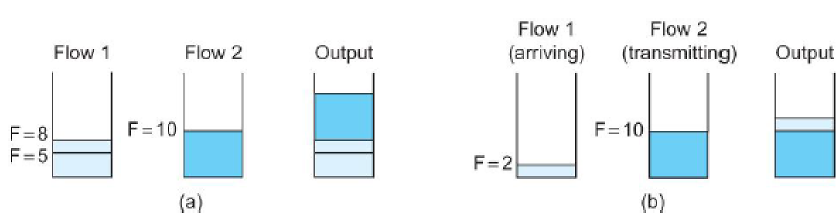
- Within each individual priority queue, the packets are still managed strictly in a FIFO manner

- **Fair queuing**:



Round-robin service of four flows at a router

  - Fair queuing was designed to address the main flaw of FIFO: its **inability to distinguish between different traffic sources or flows**
  - The algorithm creates and maintains a separate queue for every individual flow currently being handled by the router
  - The router then services these individual queues using a **round-robin** method
  - Packets can be of different lengths, so a simple round-robin approach could unfairly favor flows with larger packets
  - To ensure true fairness, FQ attempts to simulate "bit-by-bit round-robin" where the router conceptually transmits one bit from flow 1, then one bit from flow 2, and so on,
  - Since interleaving bits physically is not feasible, the FQ mechanism calculates the exact time a packet would finish transmitting if bit-by-bit round-robin were used
  - These calculated finishing times are treated as **timestamps**, and the routers always sequences the packets by **transmitting the one with the lowest timestamp first**
    - The packet that should finish transmission before all others

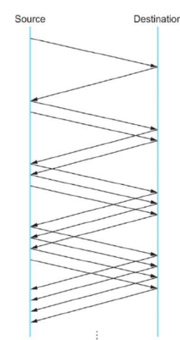


Example of fair queuing in action: (a) packets with earlier finishing times are sent first; (b) sending of a packet already in progress is completed

TCP Congestion Control Mechanisms

- TCP congestion control was introduced to the internet in the late 1980s by Van Jacobson to combat "congestion collapse", a situation where hosts **rapidly retransmitted dropped packets**, causing even more severe network congestion
- The core philosophy of TCP congestion control is **self clocking**, where the source **determines how much network capacity is available**, and uses the arrival of **ACK packets** as a **signal that it is safe to insert new packets into the network**.
- There are three primary mechanisms that work together to manage this process:
    - **Additive Increase Multiplicative Decrease (AIMD)**
      This is the fundamental algorithm TCP uses to find the right sending rate by constantly testing the networks's limits. It relies on a state variable called the CongestionWindow, which **limits how muck unACKed data the source can have in transit**. The effective transmission window is the minimum of this CongestionWindow and the receiver's AdvertisedWindow.
        - **Multiplicative decrease:**
          TCP assumes that packet timeouts are caused by network congestion, not transmission errors. Every time a timeout occurs, the source immediately cuts the CongestionWindow in half. It can decrease all the way down to a single packet (Maximum Segment Size/MSS)
        - **Additive increase:**

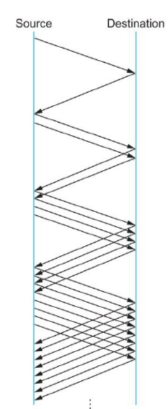
          If the network is stable and a full window's worth of packets is successfully acknowledged within one Round Trip Time (RTT), TCP cautiously increases the CongestionWindow by one packet. In practice, it adds a fraction of an MSS for every individual ACK received.



Packets in transit during additive increase, with one packet being added each RTT.

    - **Slow Start**

      While additive increase is great for maintaining a steady flow, it is too slow when starting a brand new connection.
        - **Exponential growth:**
          Slow start begins by **setting the CongestionWindow to exactly one packet**. For every ACK received, it adds one packet to the window. Because two ACKs return after sending two packets, the window size effectively doubles every RTT



Packets in transit

        - **The threshold:**
          Slow start runs until it hits a packet loss. When a timeout occurs, TCP divides the window in half and saves that value as the CongestionThreshold. The window is then reset to one packet and slow start begins again. Once the exponentially growing window hits that CongestionThreshold, TCP switches back to the safer, linear Additive Increase phase.
    - **Fast Retransmit and Fast Recovery:**
      Traditional timeouts create long periods where the connection goes completely dead. These two enhancements were added to speed up error recovery:
        - **Fast retransmit:**
          If a packet arrives out of order at the destination, the receiver immediately sends a "duplicate ACK" for the last correct packet it saw. If the sender receives three of these duplicate ACKs in a row, it assumes a packet was lost and retransmits it immediately, without waiting for the timeout timer to expire.
        - **Fast recovery:**
          Because fast retransmit catches the error early, TCP doesn't need to drop the window all the way back to one packet and restart slow start. Instead, fast recovery bypasses the slow start phase entirely, cutting the window in half and immediately resuming additive increase using the ACKs that are still arriving

Congestion Avoidance Mechanisms

- Congestion avoidance predicts when a congestion is about to happen and reduces the rate at which hosts send data just before routers are forced to discard packets.
- There are three main congestion avoidance mechanisms:
    - **DECbit (Explicit notification):**
      This mechanism was originally developed for the Digital Network Architecture (DNA) and splits the responsibility for congestion control between the routers and the end nodes
        - **The router's role:**
          Each router monitors its own queue length. If the average queue length is >= 1, the router explicitly sets a binary "congestion bit" in the header of the packets flowing through it
        - **The host's role:**
          The destination host copies this congestion bit into the ACK packet it

sends back to the source. Then, the source tracks what fraction of the last window's worth of packets had this bit set

- If **less that 50%** of the packets have the bit set, the sources **increases** its **congestion window by one packet**
- If **50% or more** of the packets have the bit set, the sources **decreases its congestion window to 0.875 times its previous value**. The 50% threshold is mathematically chosen because it corresponds to the peak of the power curve.

○ **Random Early Detection (RED):**

Unlike DECbit which explicitly marks a packet, RED implicitly notifies the source of impending congestion by intentionally dropping a packet earlier than it has to. Because RED drops a packet before the buffer space is completely exhausted, the TCP source detects the loss (via timeout or duplicate ACKs) and slows down, hopefully preventing the need to drop many more packets later.

RED decides which packets to drop based on a calculated average queue length (AvgLen) and two thresholds (MinThreshold and MaxThreshold)

AvgLen = (1 − Weight) × AvgLen + Weight × SampleLen

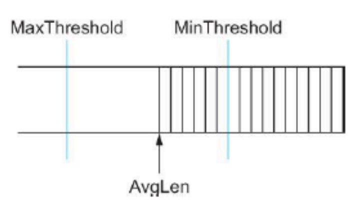
where 0 < Weight < 1 and SampleLen is the length of the queue when a sample measurement is made.

- If **AvgLen <= MinThreshold** ⇒ the packet is safely queued
- If **MinThreshold < AvgLen < MaxThreshold** ⇒ the router calculates a drop probability (P) and randomly drops the arriving packet based on that probability

  P is a function of both AvgLen and how long it has been since the last packet was dropped.

  Specifically, it is computed as follows:

  - TempP = MaxP × (AvgLen − MinThreshold)/(MaxThreshold − MinThreshold)
  - P = TempP/(1 − count × TempP)



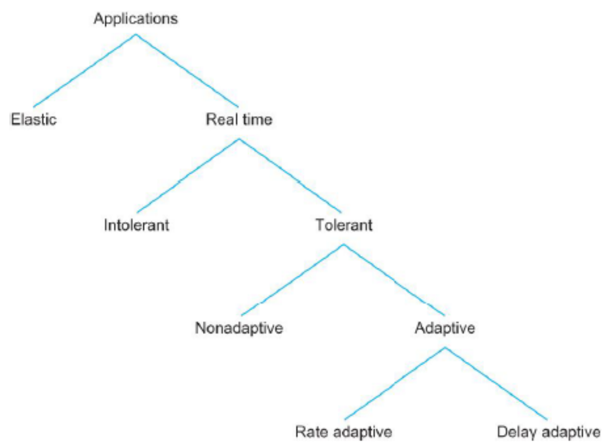
RED thresholds on a FIFO queue

- If **AvgLen >= MaxThreshold** ⇒ the router immediately drops the arriving packet

- **Source-based Congestion Avoidance:**
  This approach is driven entirely by the end hosts observing the network, without explicit assistance from the routers. The source watches for signs that router queues are building up by monitoring measurable increases in the RTT of the packets it sends. There are two distinct algorithms mentioned for doing this:
  - **Algorithm 1 (Average RTT):**
    The congestion window increases normally, but every two round-trip delays, the algorithm checks if the current RTT is greater than the average of the minimum and maximum RTTs seen so far. If it is greater, the sources decreases the congestion window by one-eighth
  - Algorithm 2(Cross-Calculation):
    Every two round-trip delays, the decision to change the window size is based on the product of the changes in both the window size and the RTT:
    $(CurrentWindow - OldWindow) \times (CurrentRTT - OldRTT)$
    - If the result is **positive**, the source **decreases** the **window size by one-eighth**
    - If the result is **0 or negative**, the source **increases** the **window by one maximum packet size**

Quality of Services

- Quality of Service is a network capability designed to support real-time applications that require strict timeliness, which traditional "best-effort" network models cannot guarantee.
- Applications like telephone conversations, video streaming, and industrial robot control need assurances that data will arrive on time to be useful
  - **Taxonomy of real-time application**

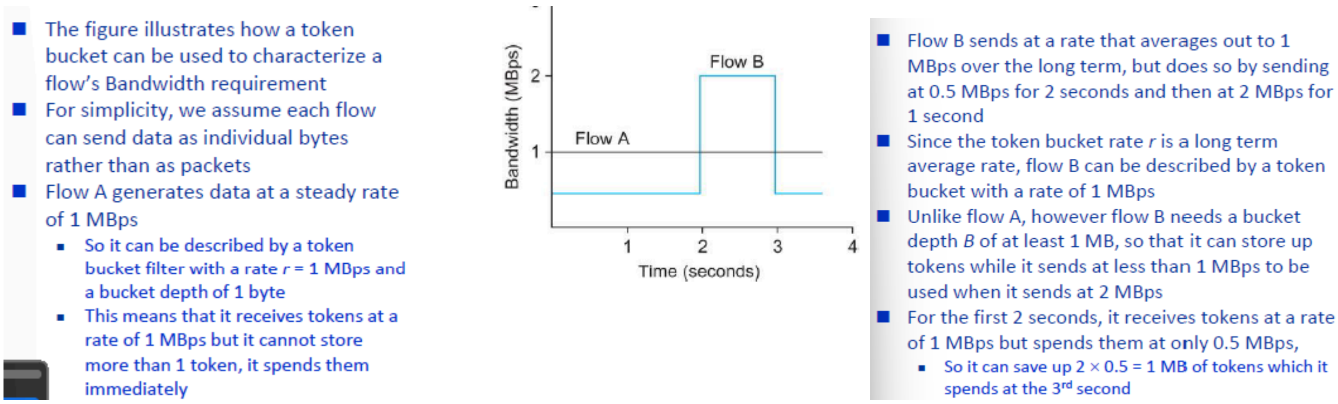


Real-time applications are categorized by how they handle less-than-ideal network conditions:

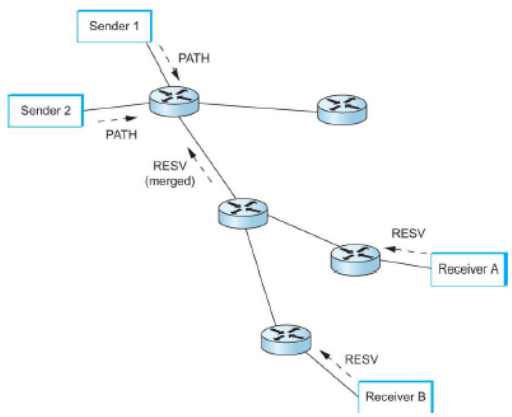
- **Tolerance to loss:**
  Applications can be **tolerant**, i.e. missing an audio sample can be interpolated without ruining the call,
  or **intolerant**, i.e. losing a command packet for a robot arm is unacceptable
- **Adaptability:**
  Applications can be **delay-adaptive**, adjusting the playback point to accommodate varying network delays,
  or **rate-adaptive**, adjusting the coding bit rate, such as video quality, based on available bandwidth

- **Fine-grained QoS: Integrated Services (IntServ/RSVP)**
  Integrated services is an architecture that allocates resources to individual, specific application flows. It ensures QoS through several mechanisms such as:
  - **Service classes:**
    Offers "guaranteed service" (assuring a specific maximum delay for any packet) and "controlled load service" (emulating a lightly loaded network even during heavy overall traffic)
  - **Flowspec:**
    To reserve resources, an application provides a Flowspec consisting of a TSpec (traffic characteristics) and an RSpec (the requested service). Because bandwidth varies constantly, the TSpec uses a Token Bucket Filter model, defined by a **token accumulation rate (r)** and a **bucket depth (B)** to describe how much data the flow will send and how bursty it might be.

To be able to send a byte, a token is needed (to send a packet of length n, n tokens are needed). Tokens are accumulated at a rate of r per second, and no more than B tokens can be accumulated.



- **Resource Reservation Protocol (RSVP):**

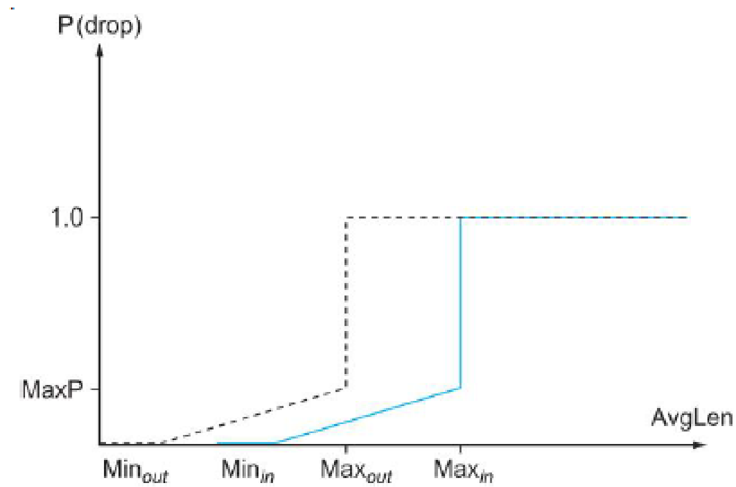


This is the setup protocol used to establish reservations across the network routers. It uses PATH messages sent from the sender to the receiver to map the route and deliver the TSpec, followed by RESV messages sent back up the path from the receiver to establish the actual resource reservations at each router. It relies on "soft state" to maintain network robustness if routers crash.

- **Coarse-grained QoS: Differentiated Services (DiffServ):**
  Managing individual flow reservations (IntServ) can be highly complex for large networks, so Differentiated Services allocates resources to a small number of broad traffic classes, i.e. premium vs. best-effort
  - **Packet Marking:**
    Rather than using a complex setup protocol, packets identify their

class via a specific bit in the packet header, which is typically set at an administrative boundary (like the edge of an ISP's network).

- **Per-Hop Behaviors (PHBs):**
  Routers apply standardized behaviors to these marked packets.
  - ***Expedited Forwarding (EF):*** Guarantees that premium packets are forwarded with minimal delay and loss.
  - ***Assured Forwarding (AF):*** Uses variations of Random Early Detection (RED), such as "RED with In and Out" (RIO). It utilizes two separate drop probability curves, meaning "out" (lower priority) packets will be discarded before "in" (higher priority) packets when congestion occurs.



RED with In and Out drop probabilities

Application Layer

Naming Issues

Naming issues refer to the fundamental challenge of how to properly identify, locate, and address the specific services, data, or application processes you want to communicate with across the network.

Human Names vs. Machine Addresses

Just as people use identifiers like names or passport numbers, entities on the internet need unique identifiers to communicate. The issue is that humans and machines require different type of identifiers:

- Humans prefer to use readable names such as www.yahoo.com
- Internet hosts and routers requires 32-bit IP addresses to properly address and route data packets (datagram) through the network

Addressing the Specific Process (IP + Port)

Even if you know the IP address of the destination computer (host), that information alone is not enough to deliver a message to the correct application

- **Problem:** a single host computer can run many different networked processes at the same time, the network needs a way to pinpoint the exact destination process
- **Solution**: a complete identifier must include both the **host's IP address** and a specific **port number**. The port number ensures the receiving host knows exactly which local process should receive the message, i.e. using port 80 for an HTTP web server or port 25 for a mail server

The Translation Solution (DNS)

To bridge the gap between the names humans prefer and the IP addresses machines require, the Internet uses the Domain Name System (DNS). DNS acts as a massive, distributed, and hierarchical database that translates (maps) human-readable hostnames into machine-routable IP addresses.

Application Architectures

Client-Server Architecture

In this model, the roles of the computers on the network are strictly separated into "servers" (providers of data/service) and "clients" (consumers of data/service).

- **Server**:
  - The server must be an "always-on" host.
  - possesses a permanent IP address so clients can always find it.
  - To handle heavy loads, servers often utilize "server farms" (clusters of multiple servers) for scaling.
- **Clients**:
  - Clients communicate solely with the server. They do not communicate directly with each other.
  - Unlike the server, clients can be intermittently connected to the network and may have dynamic (changing) IP addresses.

Peer-to-Peer Architecture

In a pure P2P model, the rigid separation between client and server is removed. Every participant (peer) can act as both a client and a server.

- **No Central Server**: There is no "always-on" central server required to facilitate the exchange of data.
- **Direct Communication**: Arbitrary end systems (the peers) communicate directly with one another.
- **Dynamic Connectivity**: Peers are often intermittently connected to the network and their IP addresses can change frequently.
- **Pros and Cons**: Because every new peer adds both demand and service capacity to the network, this architecture is highly scalable. However, the lack of centralized control makes it difficult to manage. An example of a pure P2P architecture is Gnutella.

Hybrid (Client-Server + P2P)

Many applications use a hybrid approach to combine the strengths of both architectures

- **File Sharing** (e.g., Napster):
  Uses a centralized server for the "search" function (peers register their content and query the central server to locate files), but the actual file transfer is done directly P2P.
- **Instant Messaging**:
  Uses a centralized server for "presence detection" (users register their IP with the server to find out which buddies are online), but the actual chatting happens directly P2P between the two users.

Lookup Systems

Lookup systems are mechanisms used within a network to identify, locate, and resolve where specific data, services, or users reside. These systems are divided into two primary architectural models:

Hierarchical: DNS

- The Domain Name System (DNS) is the definitive example of a hierarchical lookup system. It acts as a distributed database designed to translate human-readable hostnames (like www.yahoo.com) into the 32-bit IP addresses that machines use to route traffic.
- If DNS were a single, centralized database, it would suffer from a single point of failure, massive traffic bottlenecks, and an inability to scale globally. Therefore, DNS is strictly organized into a distributed hierarchy:
    - **Root DNS server:**
        - The very top of the hierarchy
        - consisting of 13 root name servers worldwide
        - contacted if a local name server cannot resolve a name
    - **Top-level domain servers:**
        - Just below the root
        - These servers are responsible for top-level domains like .com, .org, .edu, and country codes like .uk or .fr
    - **Authoritative DNS servers:**
        - Maintained by the specific organizations or their service providers to provide the final authoritative mapping of their hostnames, like their web or mail servers, to IP addresses
    - **Local name servers:**
        - While not strictly part of the global hierarchy, every ISP provides a local name server (default proxy)
        - When you make a request, it goes to the local server first, which then traverses the hierarchy on your behalf using either iterated queries or recursive queries
            - **Recursive query**: puts burden of name resolution on contacted name server
            - **Iterated query**: contacted server replies with name of server to contact → "I don't know this name, but ask this server"

Peer-to-Peer: Unstructured vs. Structured

In P2P architecture, there is no "always-on" central server holding the directory. Instead, arbitrary end systems (peers) communicate directly to locate and transfer files. The lookup mechanism in P2P can be implemented in different ways:

Unstructured P2P

- This represents a "Pure P2P architecture," with Gnutella being the classic example mentioned in the slides.
- In an unstructured system, peers form arbitrary, randomized connections with one another. Because there is no organized "map" of where files live, finding a file usually requires flooding a search query across the network from peer to peer.
- While this makes the network highly scalable and robust, it is also "difficult to manage" and search lookups can be highly inefficient.

Structured P2P

- The lecture references this approach when mentioning the DHT (Distributed Hash Table) paradigm.
- Unlike unstructured networks, a structured P2P system uses cryptographic math (hashing) to organize peers and data into a specific topology (like a logical ring).
- The lookup system assigns specific peers to hold specific data directories. If a file exists in the network, a DHT guarantees that the search query will efficiently jump directly to the peer holding the data, completely avoiding the messy "flooding" method used by unstructured systems.

API and Transport Layer Services: TCP, UDP, RTP

API & Sockets

To communicate over the internet, an application must pass its data down to the network. The interface that connects the application layer to the transport layer is called an **API (Application Programming Interface)**

- **Sockets** → the specific API used for the internet
  - Analogi: socket itu mirip pintu
  - A sending process shoves its message out the door and relies on the transport infrastructure on the other side to carry it to the receiving process's door
- **Division of Control:**
  - The application developer has full control over the process and what goes into the socket, but everything on the other side of the socket, such as

transport buffers, variables, and the physical internet connection, is controlled by the computer's OS

- **Addressing:**
  - To make sure the message goes to the right "door" (socket) on the receiving end, the socket identifier requires two pieces of information, the **IP address** (to find the host computer), and the **Port Number** (to find the specific application process running on that computer)

The Network Service Gap

The raw Internet provides a **"Best-Effort" Datagram Service**. It delivers data in chunks (packets) as fast as possible, but the service is quite "shoddy": packets can be lost, delivered out of order, or duplicated, and there are absolutely no guarantees on delivery time. However, applications usually want a neat, ordered byte-stream with reliable data delivery. The Transport Layer solves this by "packaging" the shoddy network service into specific protocols tailored to different application needs.

Transport Layer Protocols: TCP, UDP, RTP

Depending on what the application requires, it will choose one of the following transport services:

	TCP (Transmission Control Protocol)	**UDP (User Datagram Protocol)**	**RTP (Real-time Transport Protocol)**
Service type	Provides a reliable, "virtual pipe" connection	Provides an unreliable, datagram service	Unreliable, real-time service
How it works	Connection-oriented, meaning it establishes and maintains a connection state between the two end hosts before sending data	UDP does the bare minimum. it adds almost no packaging to the raw internet service other than the ability to multiplex/demultiplex using port numbers	RTP is actually "wrapped" over UDP. It takes the speed of UDP and adds a few specialized features specifically designed for real-time media
Features	Provides an ordered byte-stream, ensures 100% reliable data delivery by re-ordering mixed up packets, recovering lost data, and deleting duplicates. Also implements flow and congestion control so it doesn't overwhelm the	doesn't establish a connection, doesn't guarantee delivery, and doesn't reorder packets. It simply pushes the data out and leaves all error-handling to the application itself	It adds an ordered media stream, loss detection, and crucial timestamps. These timestamps assist the receiving application in syncing and playing back audio/video smoothly. Notably, while it detects loss, it makes no effort to recover lost packets,

	receiving computer or the network routers		because in live audio/video, waiting for a delayed retransmission is worse than just skipping a slightly glitched frame.
Used for	- File transfers, - Web browsing (HTTP) - Email (SMTP)	Applications where speed is more important than perfect accuracy, or applications that want to build their own custom error-handling mechanisms.	- Streaming audio/video - VoIP (internet telephony) - interactive multimedia gaming

Example Applications and Application Layer Protocols

HTTP

- HTTP is the application layer protocol for the World Wide Web, operating on a client/server model where the browser is the client and the Web server provides the objects.
- **The basic characteristics of HTTP:**
  - uses **TCP** (usually on port 80) to transfer data
  - "**stateless**," meaning the server maintains no past history or information about previous client requests, which keeps the protocol relatively simple
- **HTTP connections**
  - **Nonpersistent HTTP (HTTP/1.0)**
    - **At most, one object is sent over a single TCP connection**
    - Downloading multiple objects requires opening multiple TCP connections, taking 2 Round Trip Times (RTTs) per object
  - **Persistent HTTP (HTTP/1.1)**
    - The default mode where the server leaves the TCP connection open after sending a response, allowing multiple objects to be sent over a single connection
    - It supports "pipelining," allowing clients to send requests for referenced objects as soon as they are encountered
- **HTTP messages**
  - **Requests**: Contain a request line (methods like GET, POST, HEAD, PUT, DELETE), header lines, and sometimes an entity body
  - **Responses**: Contain a status like, i.e. 200 OK, 301 Moved permanently, 404 not found, header lines, and the requested data
- **User-Server State (Cookies)**

    - Remember: HTTP is stateless → so, sites use cookies to remember users for authorization, shopping carts, or recommendations
    - This involves a cookie header in the response, a cookie header in future requests, a cookie file saved on the user's browser, and a backend database at the website.
- **Web caching (proxy servers)**
    - Caches are usually installed by ISPs to satisfy client requests without involving the origin server, reducing response times, and network traffic
    - Caches use a "Conditional GET" (If-modified-since) to check if their cached copy is still up-to-date → if it is, the server responds with a '304 not modified' to save bandwidth.

Electronic Mail

The email system consists of three major components: User agents (mail readers like outlook), mail servers (which hold mailboxes and message queues), and the SMTP protocol

SMTP (Simple Mail Transfer Protocol)

- Uses TCP (port 25) to reliably transfer messages directly from the sending mail server to the receiving mail server
- It operates in three phases: handshaking (greeting), transfer of messages, and closure.
- **Crucial constraint:** SMTP requires messages to be in strict 7-bit ASCII format.
- **SMTP vs. HTTP:** SMTP is a "push" protocol (the sender pushes data to the receiver), whereas HTTP is a "pull" protocol (the client pulls data from the server).

Message Formats and MIME (Multimedia Mail Extension)

- Standard text messages follow the RFC 822 format (containing headers like To:, From:, Subject:, and a blank line before the body).
- Because SMTP requires 7-bit ASCII, sending multimedia (images, audio) requires MIME (Multimedia Mail Extension). MIME encodes non-text data into base64 ASCII and adds content-type declarations, i.e., image/jpeg, so the receiver knows how to decode it.

Mail Access Protocols

- **Remember**: **SMTP** is only used for the **delivery/storage** of **mail to** the **receiver's server**.
- To retrieve mail from your server to your local device, you must use a Mail Access Protocol like **POP3** (handles basic authorization and download), **IMAP** (allows

complex manipulation of stored messages on the server), or **HTTP** (used by web-based mail like Gmail).

Content Distribution Networks

- A Content Distribution Network (CDN) is a geographically distributed group of servers that work together to provide fast delivery of Internet content.
    - **Simple Analogy:** Imagine a bakery in New York (the *Origin Server*). If a customer in Tokyo wants to buy bread, shipping it directly takes a long time. To solve this, the bakery opens local warehouses (*Edge Servers*) around the world. Now, the customer in Tokyo just gets their bread from the local Tokyo warehouse, drastically reducing the delivery time.
- CDNs rely heavily on the concept of **Caching** (which you learned about in the Web Caching/Proxy section):
    - **Origin Server:** This is the primary server where your website's original files are hosted.
    - **Edge Servers (PoPs - Points of Presence):** These are the CDN's secondary servers scattered across the globe.
    - **The Process:** When a user in Europe requests to load a website whose Origin Server is in the US, the request doesn't travel all the way to the US. Instead, the closest CDN Edge Server in Europe intercepts the request and delivers a saved copy (cache) of the website directly to the user.
- CDNs are primarily used to cache and deliver **static content** (files that are large or don't change uniquely for every single user), such as:
    - Images, videos, and media streams.
    - Stylesheets (CSS), JavaScript (JS) files, and fonts.
    - Static HTML pages and downloadable files (like software patches).
- Four core benefits of using a CDN:
    - **Reduced Latency (Faster Load Times):**
      By decreasing the physical distance between the user and the server, data doesn't have to cross oceans. The delay (latency) drops significantly, and pages load much faster.
    - **Reduced Origin Server Load & Bandwidth Costs:**
      Because the majority of traffic (often 80%+) is handled by the CDN's Edge Servers, your main Origin Server is protected from traffic spikes and crashes. It also heavily reduces the bandwidth costs you pay for your primary hosting.
    - **Increased Reliability and Redundancy:**
      CDNs have a massive network of servers. If one Edge Server goes offline due

    to hardware failure, user traffic is automatically rerouted to the next closest available operational server.
  - **Enhanced Security (DDoS Protection):**
    CDN servers are designed to absorb massive amounts of traffic. They act as a shield in front of your Origin Server, blocking malicious traffic and absorbing Distributed Denial of Service (DDoS) attacks before they can take your actual website offline.
- Almost all major global platforms (like Netflix, YouTube, Facebook, and Amazon) utilize massive CDN infrastructures. Some of the most popular commercial CDN providers include:
  - Cloudflare
  - Akamai
  - Amazon CloudFront
  - Google Cloud CDN

Wireless Network

Introduction and Characteristics of Wireless Networks

Elements of a Wireless Network

A wireless network relies on three main components to function:

- **Wireless hosts:**
  - Wireless hosts are the **end devices** that run applications, such as laptops and smartphones.
  - A wireless host can be mobile or completely stationary
- **Base station:**
  - Base stations are **infrastructure elements**, such as cell towers or 802.11 Access Points (APs), that are typically connected to a larger wired network
  - Acts as relays, which are responsible for sending packets back and forth between the wired network and the wireless hosts located within their specific coverage "area"
- **Wireless link:**
  - Wireless link is the radio medium used to connect mobile devices to the base station, or sometimes used as a backbone link
  - Multiple access protocols coordinate how these links are accessed, supporting various data rates and transmission distances

Network Modes

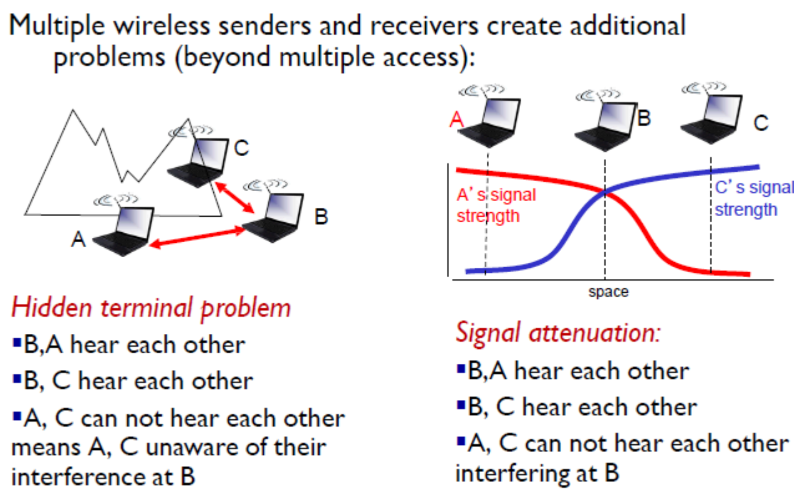
Wireless networks generally operate in one of two modes:

- **Infrastructure mode**
  - Base station is the central hub that connects mobile devices to the wired network
  - When a mobile device moves out of range of one base station and into the range of another, a "handoff" occurs, changing which base station provides the connection
- **Ad Hoc mode**
  - This mode operates entirely without base stations
  - Nodes can only transmit directly to other nodes within their immediate link coverage
  - To communicate further, the nodes must organize themselves into a network and route data among themselves

Wireless Link Characteristics

- **Decreased signal strength (path loss)**: Radio signals naturally attenuate (weaken) as they propagate through space and matter
- **Interference from other sources**: Wireless networks utilize standardized frequency bands (like 2.4 GHz) that are shared by many other devices, including cordless phones and even electrical motors, which can easily cause interference
- **Multipath propagation**: Radio signals reflect off physical objects and the ground, meaning different parts of the same signal can arrive at the destination at slightly different times, creating "ghost" signals that confuse the receiver

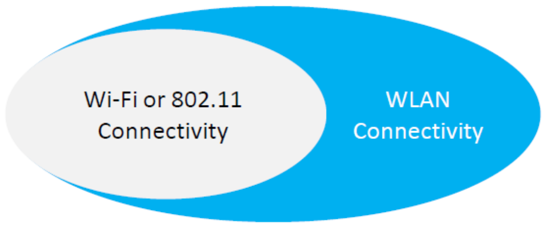
The Hidden Terminal Problem



- Imagine three nodes in a row: A, B, and C.
- Node A and Node B are close enough to hear each other. Node B and Node C are also close enough to hear each other.
- However, due to distance or a physical obstacle (like a mountain), Node A and Node C **cannot** hear each other.
- If Node A and Node C both attempt to transmit data to Node B at the exact same time, their signals will collide at Node B. Because A and C cannot hear one another, they are completely unaware that they are interfering with each other's transmission.

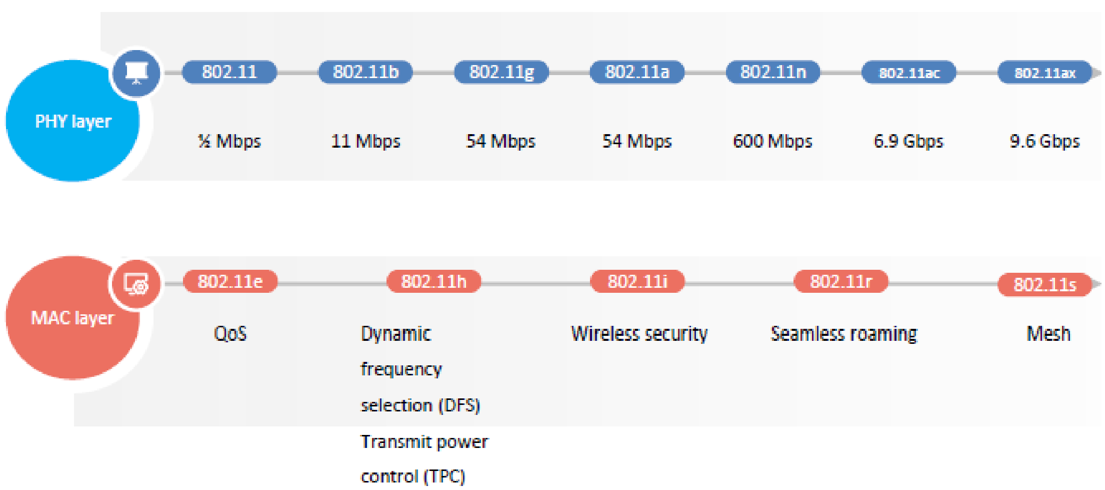
Wi-Fi Technology and IEEE 802.11 Architecture

WLAN vs. WiFi



- **WLAN (Wireless Local Area Network)**: is the general concept of combining computer networks with wireless communication technologies to allow users to move around without losing their connection
- **IEEE 802.11**: is the actual technical standard for WLANs created by the Institute of Electrical and Electronics Engineers
- **WiFi**: is a trademark owned by the Wi-Fi alliance. Essentially, IEEE 802.11 is the theoretical standard, while Wi-Fi is the physical certified implementation of that standard

The IEEE 802.11 Family and Frequencies



The 802.11 standard has evolved significantly over the years, operating primarily on two frequency bands: 2.4 GHz and 5 GHz

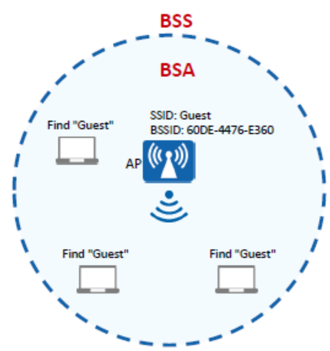
- **Frequency channels**: The 2.4 GHz band is crowded and divided into 14 overlapping channels, with only a few (typically 1,5,9,13) being non-overlapping. The 5 GHz band provides a richer spectrum with more non-overlapping channels
- **Generations**: the standard progressed from the original 802.11 (2 Mbps) through 802.11b, a, and g. Modern Wi-Fi generations include:
    - **Wi-Fi 4 (802.11n)**: Reaches 600 Mbps

    - **Wi-Fi 5 (802.11ac)**: Reaches 6.9 Gbps
    - **Wi-Fi 6 (802.11ax)**: Reaches 9.6 Gbps using both 2.4 GHz and 5 GHz
- **MAC Enhancements**: Beyond speed, the MAC layer also received updates over time, adding features like QoS (802.11e), wireless security like WPA2/WPA3 (802.11i), and seamless roaming (802.11r)

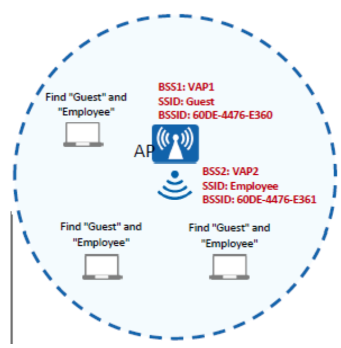
The Architecture Framework of an 802.11 Network

The structural architecture of an 802.11 network is built using several key components:

- **STA (Station)**: Any device equipped with a wireless network interface controller (WNIC). This includes Access Points (APs) as well as clients like laptops and smartphones
- **BSS (Basic Service Set)**: this is the fundamental building block of a WLAN. An infrastructure BSS consists of one central, fixed AP and multiple mobile STAs distributed around it. (there is also an independent BSS or IBSS, which is an ad hoc network without an AP)



- **BSA (Basic Service Area)**: The physical, geographical coverage area provided by the AP in a BSS. STAs can freely enter or leave this coverage bubble
- **BSSID and SSID (Naming the Network)**
    - **BSSID (Basic Service Set Identifier)**: this is the true identity of the AP, typically its raw MAC address
    - **SSID (Service Set Identifier)**: Because humans cannot easily memorize MAC addresses, the SSID is a customizable character string used as the human-readable name of the network, i.e. "Guest" or "Employee"



- **VAP (Virtual Access Point)**: A single physical AP router can be configured with multiple VAPs. This allows one router to project multiple different BSSs (and therefore multiple SSIDs, like a "Guest" network and an "Admin" network) in the same physical airspace
- **ESS (Extended Service Set) & DS (Distribution System)**: When a coverage area needs to be larger than what one AP can provide, **multiple BSSs** are **connected** together to form an **ESS**. The **APs in an ESS** are **linked** together **by** a backbone network called the **Distribution System (DS)**, which can be wired or wireless. This setup allows devices to "roam" smoothly from one AP's coverage area to another while maintaining their connection

MAC Protocol and IEEE 802.11 Data Transmission

Passive and Active Scanning

Before the host can transmit data, it must find and associate with an Access Point (AP). The 802.11 standard defines two ways to do this:

- **Passive scanning**:
  - The host listens for “beacon frames” that are periodically broadcasted by the APs.
  - Once the host hears a beacon from the desired AP, it sends an Association Request frame to that AP, and the AP replies with an Association Response frame to establish the connection
- **Active scanning**:
  - The host actively broadcasts a “Probe Request” frame to all APs in the area.
  - Available APs respond with “Probe Response” frames.
  - The host then selects an AP and goes through the Association Request/Response process.

The MAC Protocol: CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

- To **prevent multiple devices from talking over each other and causing data collisions**, 802.11 uses **CSMA/CA protocol**
- Wired networks use Collision Detection (CSMA/CD), but wireless network cannot
- In wireless, its difficult for a device to receive and sense collisions while it is transmitting due to fading signals, and it cant sense all collisions due to the hidden terminal problem ⇒ therefore, 802.11 focuses on avoiding collisions before they happen
- How it works:
  - The sender listens to (senses) the channel. If it is idle for a specific duration called DIFS, it transmits the entire frame.
  - If the channel is busy, the sender starts a random backoff timer. This timer counts down only when the channel is idle.
  - When the timer expires, the sender transmits.
  - If the receiver successfully gets the frame, it waits for a short duration called SIFS and sends an ACK (Acknowledgment) back to the sender. If the sender does not receive this ACK, it assumes a collision occurred, increases its random backoff interval, and tries again.

RTS/CTS to Solver Hidden Terminal Problem

Even with CSMA/CA, long data frames can still collide if two senders cannot hear each other (hidden terminal problem) ⇒ 802.11 solves this by allowing a sender to "reserve" the channel before sending a massive chunk of data

- **RTS (Request-to-Send)**: The sender first transmits a very small RTS packet to the AP. Even if RTS packets from two hidden terminals collide, they are so small that the network penalty is minimal
- **CTS (Confirmation-to-Send)**: Upon receiving RTS, the AP broadcasts a CTS packet ⇒ all nodes in the area (even the hidden ones) hear it. This tells the original sender to proceed with transmitting its DATA frame, and it tells all other stations to defer their transmission until the process is complete and the ACK is sent

802.11 Frame Addressing

- A standard ethernet frame only needs two MAC addresses (source and destination), but an 802.11 frame is more complex because it must travel through the air to an AP before hitting the wired router
- The 802.11 header contains slots up to **four MAC addresses**:
    - **Address 1**: The MAC **address** of the wireless host or AP that is **meant to directly receive this frame over the air**
    - **Address 2**: The MAC **address** of the wireless host or AP that is **transmitting this frame into the air**
    - **Address 3**: The MAC **address of the ultimate destination router interface** to **which the AP is attached**
    - **Address 4**: This slot is rarely used in standard infrastructure networks; it is **reserved** only **for ad hoc mode**

Cellular Networks: From 2G to 4G LTE

Basic Components of a Cellular Network

- **Cell**: A specific geographical region covered by the network
- **Base Station (BS)**: Similar to a WiFi Access Point, this is the tower that mobile users attach to through the "air-interface" (the physical and link-layer protocol between the mobile and the BS)
- **Mobile Switching Center (MSC)**: The central hub that manages call setup, handles user mobility, and connects the local cells to the broader wired public telephone network

- **First Hop Spectrum Sharing**: To allow many mobile phones to talk to a single base station at the same time, networks use techniques like **FDMA/TDMA** (dividing the spectrum into frequency bands and time slots) or **CDMA** (Code Division Multiple Access)

2G Architecture: The Voice Era

- 2G network was built primarily for voice communication
- It routes calls from mobile subscribers through a Base Transceiver Station (BTS) and a Base Station Controller (BSC)
- The data is then pushed to the Mobile Switching Center (MSC) and a Gateway MSC, which finally connects the call to the traditional Public Telephone Network

3G Architecture: Adding Parallel Data

- As mobile internet became a necessity, 3G was introduced to handle both voice and data
- **Parallel concept**: The key insight of 3G is that it leaves the existing 2G voice network (the MSC and Public Telephone connection) mostly unchanged. Instead, it builds a brand new data network that operates in parallel to the voice network
- **New Components**: For data, the radio signal goes from the cell tower to a Radio Network Controller, and is then routed through two new core network components:
    - SGSN (Serving GPRS Support Node)
    - GGSN (Gateway GPRS Support Node), which connect the user to the Public Internet

4G LTE Architecture: The "All-IP" Core

- 4G LTE completely revolutionized the architecture by throwing away the "parallel networks" concept of 3G.
- **Unified Traffic:** In 4G, there is **no separation between voice and data**. Voice calls are simply treated as data packets. All traffic is carried over an "All-IP core" network straight to the internet gateway.
- **Evolved Packet Core (EPC):** The old MSC, SGSN, and GGSN are replaced by a modern IP architecture called the **EPC**. Its main components are:
    - **eNodeB:** The new, highly intelligent base station that handles radio resource allocation and admission control directly.
    - **MME (Mobility Management Entity):** Handles idle/active transitions and sets up the data tunnels.
    - **HSS (Home Subscriber Server):** The central database containing user info and authentication data.

    - **S-GW (Serving Gateway) & P-GW (Packet Data Network Gateway):** These act as the mobile anchors and IP address allocators, tunneling the user's IP packets out to the Public Internet.
- **Tunneling (GTP):** As a user's IP packet travels from the mobile phone through the eNodeB and toward the P-GW, it is encapsulated (wrapped) inside a GPRS Tunneling Protocol (GTP) message, which is further wrapped in UDP and IP for rapid transit across the core network.

Quality of Service (QoS) in LTE

Because 4G forces everything (including sensitive voice calls) onto a single IP data network, it requires strict rules to make sure important data isn't delayed by less important data.

- LTE uses **12 QCI (QoS Class Identifier) values** to enforce priorities in the radio access network.
- **GBR (Guaranteed Bit Rate):** High-priority traffic like conversational voice (QCI 1) or live video streaming (QCI 2) receives a guaranteed minimum bit rate so it doesn't lag or drop out.
- **Non-GBR (Non-Guaranteed Bit Rate):** Lower-priority traffic like TCP-based web browsing, email, or FTP downloads (QCI 8) does not get a guaranteed speed and just uses whatever bandwidth is currently available.

Network Security

Introduction to Security Vulnerabilities

Layered Nature of Attacks

Security vulnerabilities exist at **every layer of the protocol stack**. This means that no matter how secure an application is, it can be compromised by a flaw in the underlying transport protocol or the network routing itself. Security threats are categorized by where they manifest in the networking architecture:

- **Network-layer attacks:** These focus on the fundamental IP-level vulnerabilities and attacks against routing protocols.
- **Transport-layer attacks:** These exploit vulnerabilities specific to protocols like TCP, which may involve connection hijacking or session disruption.
- **Application-layer attacks:** These target specific services or software running on top of the network, such as web servers or email applications.

IP-Level Vulnerabilities

- IP-level vulnerabilities arise largely because the Internet Protocol (IP) was designed for connectivity and efficiency rather than security
- Because IP addresses are supplied by the originating host rather than verified by the network, the protocol is inherently susceptible to exploitation

Address Spoofing

- In standard IP communication, the source host is responsible for filling in its own source IP address in the packet header
- **Vulnerability**: An attacker can easily insert a fake source IP address into the header to masquerade as a trusted host
- Consequence: Historically, many systems used "r-utilities" (such as rlogin, rsh, or .rhosts authentication) that relied solely on the source IP address to verify a user's identity. If an attacker spoofs the IP of a trusted machine, they can bypass these authentication mechanisms completely.

ARP Spoofing

- ARP (Address Resolution Protocol) is used to map an IP address to a physical hardware MAC address

- **Vulnerability:** An attacker can send malicious ARP messages to a network, linking their own MAC address to the IP address of a legitimate target, such as a server or a gateway.
- **Consequence:** Other hosts on the network will update their ARP tables and mistakenly send traffic intended for the legitimate server directly to the attacker instead.

Exploiting Protocol “Features”

- Certain inherent functions of the IP protocol were designed for network flexibility but are dangerous when exploited:
    - **Packet Fragmentation:** Attackers can manipulate fragmented packets to bypass security filters or firewalls that are not configured to properly reassemble and inspect fragments.
    - **Traffic Amplification (Smurf Attacks):** Attackers exploit broadcast-enabled networks. By sending a ping request to a broadcast address using a spoofed source IP (the victim's IP), the attacker causes every host on that network to send a reply back to the victim simultaneously, effectively flooding the victim with traffic.

ICMP Redirects

- The ICMP (Internet Control Message Protocol) is used for network diagnostics and error reporting, but it can be abused to control network traffic flow
- **Vulnerability:** An attacker can send a spoofed ICMP redirect message to a victim host.
- **Consequence:** This tricks the victim host into switching its default gateway to a machine controlled by the attacker. Once the victim’s traffic is routed through the attacker's machine, the attacker can perform a **Man-in-the-Middle (MITM) attack**, allowing them to sniff, log, or modify the victim's packets without the victim realizing their connection has been compromised.

Foundations of Cryptography (The Building Blocks)

- **Foundations of Cryptography** serves as the technical "toolkit" used to secure communications when the underlying network infrastructure itself cannot be trusted

Confidentiality (Stream & Block Ciphers)

- Confidentiality ensures that an eavesdropper cannot read the content of the message
- **Stream Ciphers:** These encrypt data one bit (or byte) at a time, often by XORing the message with a pseudorandom stream of bits generated from the secret key.
- **Block Ciphers:** These encrypt data in fixed-size blocks (e.g., 64 or 128 bits at a time). The message is processed in chunks, and the key is applied to each block.

Integrity (HMAC)

- Even if a message is encrypted, an attacker might try to alter the ciphertext during transit. Integrity ensures that the message has not been tampered with.
- **HMAC (Hash-based Message Authentication Code):** Instead of just sending a message, the sender calculates a cryptographic hash of the message combined with the secret key.
- When the receiver gets the message, they re-calculate the hash using the same secret key. If the calculated hash matches the one provided by the sender, the receiver knows the data is authentic and intact.

Authentication (HMAC and Nonce)

- Authentication confirms that the message actually came from the expected sender and is not a replayed message captured from a previous session
- **The "Nonce" Strategy:** To prevent "replay attacks", where an attacker captures a valid message and sends it again later, Alice can send Bob a **Nonce** (a "number used once"), which is a random bitstring.
- Bob must include this nonce in his HMAC calculation when he replies to Alice. Because the nonce is new and random every time, an attacker cannot simply record and replay an old HMAC result to fool Alice; the result will no longer match the "challenge" nonce Alice just sent.

Cryptographic Challenges

Scalability Challenge

- In a symmetric key system, every pair of users who wish to communicate privately must share a unique secret key.
- If a network has n users, the number of distinct secret keys required is calculated as $n(n-1)/2$
- Impact: As the number of users (n) grows, the number of keys grows quadratically ($O(n^2)$). For a small group of 10 people, you need 45 keys, which is manageable.

However, for a network of 1,000 users, you would need nearly half a million keys (499,500). This makes symmetric key management practically impossible for global-scale networks.

Key Distribution

- Even if you could manage the sheer number of keys, you face the fundamental challenge of **key distribution**, the "chicken and egg" problem of security.
- **Dilemma:** To encrypt a message, Alice and Bob must already share a secret key. But how do they share that secret key in the first place without an eavesdropper stealing it during the exchange?
- **Implication:** You cannot send the secret key over the network itself, because if the network is insecure enough to require encryption, it is also insecure enough to leak the key you are trying to transmit.

Because of these two challenges, secure protocols like TLS/SSL don't rely exclusively on symmetric keys. Instead, they use public key cryptography (asymmetric) to solve the initial key distribution problem, and then use the resulting "session keys" to perform high-speed symmetric encryption for the rest of the communication

Transport Layer Security (TLS/SSL)

Purpose and Mechanism

- **Goal:** TLS/SSL provides the three fundamental pillars of security: **confidentiality** (keeping data secret), **integrity** (ensuring data isn't altered), and **authentication** (verifying who you are talking to)
- **Placement:** It operates as a special "TLS socket layer" placed between the Application layer and the TCP layer. This clever positioning allows it to secure application data before it is handed off to TCP, requiring only small changes to the application itself

The Hybrid Cryptographic Approach

- TLS is called "hybrid" because it solves the scalability and key distribution challenges of symmetric cryptography by using both asymmetric (public key) and symmetric cryptography:
  - **Public Key Cryptography:** Used during the initial handshake to securely exchange keys without needing to share a secret beforehand

- **Symmetric Cryptography:** Once the handshake is complete and a session key is derived, the system switches to faster symmetric encryption for the actual data transmission

TLS Handshake Process

The handshake is the multi-step process used to establish a secure channel:

1. **Negotiation:** The client and server "talk" to agree on which exact cryptographic protocols and algorithms they will use
2. **Certificate Validation:** The server sends its public key certificate to the client. The client validates this certificate using a trusted Certificate Authority's (CA) public key to ensure the server is who it claims to be
3. **Key Exchange (The Challenge):** The client generates a random secret value, encrypts it with the server's public key, and sends it to the server
4. **Verification:** The server decrypts this value using its corresponding private key. This successfully proves to the client that the server is the legitimate holder of the private key
5. **Session Key Derivation:** Both parties now use this secret value to derive identical **session keys**. These keys are then used for the remainder of the session to provide high-speed, symmetric-key encryption